TRƯỜNG ĐẠI HỌC SÀI GÒN
KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO BÀI TẬP LỚN

CÔNG NGHỆ PHẦN MỀM

BrewLite

Ứng dụng đặt cà phê không dùng tiền mặt

Assignment 1 và Assignment 2
Từ yêu cầu đến triển khai

Giảng viên hướng dẫn Từ Lãng Phiêu

Nhóm .....

Lớp .....

Thành viên và mã số sinh viên

.....

.....

.....

Học kỳ I năm học 2026–2027

TP. Hồ Chí Minh, tháng 11 năm 2026

Mục lục

1 Giới thiệu đề tài **1**

1.1 Liên kết sản phẩm 1

2 Khai thác và đặc tả yêu cầu **2**

2.1 Bối cảnh và phạm vi BrewLite 2

2.2 Đặc tả yêu cầu phần mềm BrewLite 4

2.3 UC 04 Đặt và thanh toán đơn hàng 6

3 Mô hình hóa hệ thống **9**

3.1 Use case tổng quát 10

3.2 Use case trọng tâm đặt và thanh toán 11

3.3 Activity đặt và thanh toán với swimlane 12

3.4 Sequence đặt và thanh toán idempotent 13

3.5 Class diagram domain và enum 14

3.6 Máy trạng thái đơn hàng 15

4 Thiết kế kiến trúc **16**

4.1 Kiến trúc BrewLite 16

4.2 ADR 001 Monorepo npm workspaces 17

4.3 ADR 002 PostgreSQL và Prisma 18

4.4 ADR 003 Proxy cùng origin 18

4.5 ADR 004 Thanh toán mock idempotent 19

4.6 ADR 005 Delivery bằng image bất biến 19

4.7 Deployment diagram 20

5 Quản lý Agile Scrum **21**

5.1 Product Backlog 21
5.2 Kế hoạch Sprint và ánh xạ hai Assignment 22
5.3 Sprint Review và Retrospective 23

6 Hiện thực phần mềm 25
6.1 Frontend Next.js 25
6.2 Xác thực và phân quyền 25
6.3 Tạo đơn và optimistic locking 25
6.4 Thanh toán idempotent 26
6.5 Hủy và chuyển trạng thái 26
6.6 Khôi phục checkout 26

7 Kiểm thử và bằng chứng 31
7.1 Bằng chứng kiểm chứng 31
7.2 Đo hiệu năng local 33

8 Triển khai và bàn giao 34
8.1 Hướng dẫn triển khai và rollback 34
8.2 Kịch bản demo trực tiếp 36
8.3 Release notes 36

9 Truy vết và đối chiếu tiêu chí 38
9.1 Ma trận truy vết 38
9.2 Checklist bàn giao theo rubric 39

10 Tài liệu tham khảo 41
10.1 Tài liệu tham khảo 41

Chương 1

Giới thiệu đề tài

BrewLite giải quyết luồng chọn đồ uống, tùy chỉnh, đặt và thanh toán không dùng tiền mặt tại một cửa hàng cà phê. Sản phẩm sử dụng Next.js cho giao diện mobile first, NestJS cho REST API, PostgreSQL và Prisma cho dữ liệu. Thanh toán Ví và Thẻ là mô phỏng theo phạm vi đề bài.

Báo cáo kết hợp 10 task của Assignment 1 và 5 task tuần tự của Assignment 2. Nội dung trình bày phạm vi, yêu cầu có tiêu chí chấp nhận, mô hình UML, kiến trúc và ADR, kế hoạch Scrum, hiện thực, kiểm thử và bàn giao. Các link GitHub và kết quả kiểm thử là bằng chứng có thể kiểm tra; mục triển khai production chỉ được nghiệm thu sau khi có URL HTTPS hoạt động.

Công cụ hỗ trợ: nhóm sử dụng trợ lý lập trình AI (Claude Code và OpenAI Codex) để hỗ trợ viết mã, kiểm thử, rà soát code và soạn tài liệu. Mọi thay đổi đi qua commit/pull request, được kiểm chứng bằng test tự động và CI; nhóm chịu trách nhiệm về nội dung và kết quả nghiệm thu.

1.1 Liên kết sản phẩm

Repository nộp bài: https://github.com/Kenkra9999/Brew_Lite_2. Repository này nhận nguyên lịch sử commit, tag v1.0.0 và Product Backlog từ repository làm việc ban đầu https://github.com/Kenkra9999/brewlite2, nơi có pull request hoàn thiện https://github.com/Kenkra9999/brewlite2/pull/1. Báo cáo và tài liệu nguồn nằm cùng repository để truy vết thay đổi.

Chương 2

Khai thác và đặc tả yêu cầu

2.1 Bối cảnh và phạm vi BrewLite

Môn Công nghệ Phần mềm, Đại học Sài Gòn, HK I 2026–2027. Nguồn yêu cầu: `BrewLite-CongNghePhanMem.pdf` (Assignment 1, 10 task) và `Assignment2-BrewLite.pdf` (Assignment 2, 5 task). Hai tài liệu được kết hợp trên cùng sản phẩm, không thay thế Next.js, NestJS hoặc PostgreSQL.

2.1.1 Vấn đề và mục tiêu

Khách phải chờ chọn đồ uống và trả tiền tại quầy. BrewLite cho phép xem menu, chọn size/topping, đặt và trả tiền mô phỏng trước, nhận mã đơn và theo dõi quá trình pha chế. Thành công của MVP là hoàn tất luồng này với dữ liệu lưu thật, giá tính ở server, không thu tiền lặp và không âm tồn kho.

Stakeholder	Mục tiêu	Tiêu chí quan tâm
Khách hàng	Đặt nhanh, biết giá và tình trạng đơn	Giỏ rõ ràng, thử lại an toàn, chỉ xem đơn mình
Nhân viên quầy	Biết đơn đã trả tiền, cập nhật pha chế	Role STAFF và chuyển trạng thái hợp lệ
Quản lý cửa hàng / PO	Ưu tiên phạm vi, kiểm soát bán hàng	Menu, kho, mã giảm giá và demo nghiệm thu
Giảng viên / khách hàng nghiệm thu	Đánh giá quy trình và sản phẩm	Yêu cầu, UML, test, Git, CI/CD, URL hoạt động
Nhóm phát triển	Tái tạo và bảo trì hệ thống	Migration, seed, Docker, test và ADR
Đơn vị thanh toán mô phỏng	Trả kết quả thanh toán có thể kiểm chứng	Một kết quả cho mỗi Idempotency-Key

2.1.2 In scope

Menu có hình/giá; chi tiết; size S/M/L; topping; giỏ có lưu trên trình duyệt; đăng ký/đăng nhập JWT; đơn PENDING; thanh toán WALLET/CARD giả lập; xác nhận, lịch sử và theo dõi; khuyến mãi; điểm thưởng; optimistic locking tồn kho; idempotency; bàn nhân viên cập nhật trạng thái; Docker, CI, cấu hình CD, tài liệu và kiểm thử. Quầy nhân viên là phần mở rộng của Assignment 1 để minh họa state machine và đáp ứng quản lý trạng thái của Assignment 2.

2.1.3 Out of scope

Thu tiền thật, giao hàng, nhiều chi nhánh, hoàn tiền ngân hàng thật, quản lý công thức nguyên liệu, hóa đơn thuế, loyalty đổi điểm, đăng nhập mạng xã hội. Không lưu số thẻ. Tồn kho được mô hình hóa bằng số phần đồ uống còn bán được, không phải gram nguyên liệu.

2.1.4 Giả định và ràng buộc

- Một cửa hàng, đơn vị tiền VND nguyên, không có phí giao hàng hay thuế cộng thêm.
- Size S giá gốc, M +5.000, L +10.000. Topping tính theo từng ly, tối đa 4 loại; mỗi dòng 1–20 ly, mỗi đơn tối đa 20 dòng.
- Kho được giữ khi tạo PENDING; PAYMENT_FAILED giữ kho để thử lại; hủy hợp lệ hoàn kho và lượt mã đúng một lần. Chưa có tác vụ tự hết hạn đơn treo.
- WALLET giả lập có hạn mức 2 triệu/giao dịch; CARD chỉ dùng số thử nghiệm. PAID cộng floor(total/10.000) điểm một lần.
- Công nghệ bắt buộc Next.js + NestJS + PostgreSQL; dùng Prisma migration; Node 22 và Docker Compose v2.
- Dữ liệu demo và tài khoản demo chỉ phục vụ chấm bài; production thật phải bỏ tài khoản demo và cấu hình secret riêng.
- Các bằng chứng CI, production và buổi Scrum chỉ được ghi nhận khi thực sự có; không coi cấu hình pipeline là một lần deploy thành công.

2.1.5 Rủi ro ban đầu

Rủi ro	Tác động	Biện pháp
Hai khách mua phần cuối	Bán quá kho	Transaction, version, điều kiện stock >= qty, retry và test đồng thời

Rủi ro	Tác động	Biện pháp
Client lặp thanh toán	Thu tiền/điểm lặp	Unique key, khóa dòng Order, hash request, replay
Mạng mất sau khi trả tiền	Khách không biết kết quả	Giữ key và orderId trên phiên, tải lại đơn, retry cùng key
API chết giữa hai transaction thanh toán	Payment kẹt PROCESSING	Health/log; ghi rõ giới hạn mock, cần reconciliation/ outbox trước thanh toán thật
Không có tài khoản hosting/ domain	Không nghiệm thu production A2 Task 5.4	Chuẩn bị Compose production + CD + runbook, ghi trạng thái thật và chờ môi trường
Thiếu thông tin thành viên	Bìa chưa định danh	Nhóm điền `report/thong-tin.tex` trước khi nộp

2.2 Đặc tả yêu cầu phần mềm BrewLite

2.2.1 Phạm vi và thuật ngữ

Đặc tả này kết hợp Assignment 1 và 2. Must là bắt buộc MVP, Should ưu tiên kế tiếp, Could ở backlog, Won't ngoài phiên bản. PAID là kết quả mock, không phải tiền thật. Xem `context-and-scope.md` để hiểu giả định kho và giới hạn thanh toán.

2.2.2 Yêu cầu chức năng

ID	Yêu cầu	MoSCoW	Tiêu chí chấp nhận
FR-01	Xem menu	Must	GET /products trả mảng id, name, price, imageUrl, stock; UI có loading, empty, error.
FR-02	Xem chi tiết	Must	GET /products/:id trả chi tiết, sizes, toppings; id không tồn tại trả 404.
FR-03	Chọn tùy chọn	Must	S/M/L và tối đa 4 topping; UI cập nhật giá, API tự tính giá từ DB.
FR-04	Sửa giỏ	Must	Thêm, sửa qty 1–20, xóa; badge và tổng thay đổi đúng, tách dòng theo size/topping.
FR-05	Lưu giỏ	Should	Reload còn giỏ, thanh toán lỗi giữ nguyên giỏ; thành công xóa giỏ.
FR-06	Hiển thị tổng	Must	subtotal là tổng unitPrice*qty; discount không vượt subtotal; total=subtotal-discount.
FR-07	Đăng ký	Must	Email duy nhất, password có chữ và số >=8 ký tự, lưu bcrypt hash, trả JWT.

ID	Yêu cầu	MoSCoW	Tiêu chí chấp nhận
FR-08	Đăng nhập	Must	Đúng tài khoản trả JWT; sai trả 401, không tiết lộ email tồn tại.
FR-09	Phân quyền	Must	Thiếu/sai JWT không đặt đơn; khách không đọc đơn người khác; chỉ STAFF đổi trạng thái.
FR-10	Validate	Must	Client có ràng buộc form; API từ chối size sai, qty sai, giỏ trống và giá client gửi.
FR-11	Khuyến mãi	Should	WELCOME10 giảm 10% từ 50k, tối đa 20k; GIAM15K giảm 15k từ 100k; mã sai/hết hạn bị chặn.
FR-12	Tạo đơn	Must	POST /orders tạo đúng một đơn PENDING cho một lần gọi hợp lệ và snapshot tên/giá trong transaction.
FR-13	Thanh toán	Must	POST /payments hỗ trợ WALLET/CARD mock; thành công Payment SUCCEEDED và Order PAID.
FR-14	Lỗi và thử lại	Must	Thẻ 4000000000000002 bị từ chối, Order PAYMENT_FAILED; thử lại mới dùng key mới trên cùng đơn.
FR-15	Xác nhận và theo dõi	Must	Hiển thị code, status, tổng, item và lịch sử trạng thái sau thanh toán; polling không cache API.
FR-16	Lịch sử đơn	Must	GET /orders/me trả tối đa 50 đơn của người hiện tại, mới nhất trước.
FR-17	State machine	Must	Chỉ cho PEND-ING→PAID→PREPARING→READY→COMPLETED và các nhánh lỗi/hủy đã định nghĩa; sai trả 409.
FR-18	Kho đồng thời	Must	20 yêu cầu đồng thời mua sản phẩm còn 5 phần: đúng 5 đơn, kho 0, không âm.
FR-19	Loyalty	Should	Chỉ cộng floor(total/10000) sau PAID; replay không cộng lần hai; STAFF hủy PAID thu hồi điểm.
FR-20	Hủy đơn	Should	Khách hủy PENDING/PAYMENT_FAILED; chặn khi thanh toán đang chạy; hoàn kho/lượt mã đúng một lần.
FR-21	Idempotency	Must	Cùng key + cùng user/order/method trả cùng payment; khác payload trả 422; đồng thời không charge hai lần.
FR-22	Thanh toán thật	Won’t	Không kết nối ngân hàng hoặc yêu cầu thẻ thật trong phiên bản này.
FR-23	Thông báo đẩy	Could	Để backlog sau MVP; hiện dùng polling.

2.2.3 Yêu cầu phi chức năng

ID	Nhóm	MoSCoW	Tiêu chí chấp nhận
NFR-01	Hiệu năng	Should	GET /products: p95 <500ms với seed 8 món, 100 request tuần tự trên máy đo đã ghi cấu hình; không áp cho gateway giả lập.
NFR-02	Bảo mật	Must	bcrypt, JWT, validate, phân quyền; không log password/token/card, DB không public trong Compose production.

ID	Nhóm	MoSCoW	Tiêu chí chấp nhận
NFR-03	Tính khả dụng	Must	/health kiểm tra DB; lỗi API hiển thị nút thử lại; Compose đợi health và restart dịch vụ.
NFR-04	Tương thích	Must	Hoàn tất đặt đơn ở Chromium desktop và viewport 390x844; không tràn ngang.
NFR-05	Tái lập	Must	npm ci, migration/seed và docker compose up –build tái tạo được ba dịch vụ từ README.
NFR-06	Quality gate	Must	PR chạy lint, unit, integration, E2E và build; branch protection được cấu hình trước nghiệm thu merge gate.
NFR-07	Triển khai	Must	HTTPS frontend/API, image tag SHA, health smoke, migration và rollback có tài liệu; nghiệm thu chỉ khi URL thực hoạt động.
NFR-08	Truy cập	Should	Nhãn form, focus rõ, trạng thái lỗi role=alert, không truyền trạng thái chỉ bằng màu.
NFR-09	PWA	Should	Manifest và service worker; shell offline thông báo không có mạng; API và thanh toán không cache.

2.2.4 Use case tổng quát

Nguồn và bản kết xuất: `../diagrams/usecase-system.puml` và `.svg`. UC-01 xem menu/chi tiết; UC-02 quản lý giỏ; UC-03 xác thực; UC-04 đặt và thanh toán; UC-05 lịch sử/theo dõi; UC-06 cập nhật đơn (STAFF). Use case trọng tâm trong `use-case-checkout.md`.

2.2.5 Quy tắc nghiệp vụ

BR-01: tiền nguyên VND, server là nguồn giá chính thức. BR-02: kho trừ khi PENDING và hoàn khi CANCELLED. BR-03: một payment cho một idempotency key, nhiều lần thử trên cùng đơn được phép nếu lần trước FAILED. BR-04: chỉ gateway tạo PAID/PAYMENT_FAILED. BR-05: điểm thưởng và thay đổi trạng thái thanh toán nằm cùng transaction. BR-06: số thẻ chỉ ở request tới mock gateway, không lưu DB, localStorage hay log. BR-07: đơn PAID hủy tại quầy chỉ là hoàn tiền mô phỏng; chưa có ledger refund thật.

2.3 UC 04 Đặt và thanh toán đơn hàng

Thuộc tính	Nội dung
Actor chính	Khách hàng đã có giỏ
Actor phụ	Dịch vụ thanh toán giả lập
Trigger	Khách chọn Thanh toán từ giỏ

Thuộc tính	Nội dung
Tiền điều kiện	API và DB sẵn sàng, sản phẩm active; trước khi tạo đơn cần JWT hợp lệ
Hậu điều kiện thành công	Một Order PAID, Payment SUCCEEDED, kho đã giữ, điểm cộng một lần, UI hiển thị mã và xóa giỏ
Hậu điều kiện thất bại	Tạo đơn thất bại rollback toàn bộ; payment thất bại Order PAYMENT_FAILED, giỏ giữ nguyên; không charge thành công
Yêu cầu	FR-03..FR-15, FR-18..FR-21, NFR-02,03,05
Sơ đồ	usecase-checkout, activity-checkout, sequence-checkout, class-domain

2.3.1 Luồng chính

1. Khách xem menu, mở chi tiết và chọn size, topping, qty.
2. UI thêm vào giỏ và tính tổng tạm tính. Khách xem lại giỏ, nhập note/mã giảm giá nếu có.
3. Nếu chưa đăng nhập, UI chuyển sang form và quay lại checkout sau khi JWT được cấp.
4. Khách chọn WALLET hoặc CARD thử nghiệm và xác nhận.
5. UI gọi POST /orders. NestJS kiểm tra JWT, DTO, giá và kho trong transaction; optimistic lock trừ kho; kiểm tra mã, tạo snapshot items và PENDING; commit.
6. UI lưu orderId và sinh Idempotency-Key; gọi POST /payments trên chính đơn đó.
7. API khóa dòng Order, kiểm tra chủ đơn, state, payment đang xử lý và key; tạo Payment PROCESSING; commit.
8. Gọi mock gateway ngoài transaction; sau kết quả, transaction khóa Order, lưu Payment SUCCEEDED, chuyển PAID và cộng điểm.
9. API trả kết quả. UI xóa giỏ, hiện mã đơn và polling GET /orders/:id để theo dõi.

2.3.2 Luồng thay thế

A1 ở bước 3: đăng ký tài khoản rồi trở lại checkout; từ chối xác thực thì giỏ không mất.

A2 ở bước 5: hết hàng → 409; transaction rollback, giỏ giữ nguyên để sửa.

A3 ở bước 5: mã sai/hết hạn → 422; không tạo đơn, không mất kho.

A4 ở bước 8: thẻ bị từ chối → Payment FAILED, Order PAYMENT_FAILED; giỏ và orderId giữ nguyên. Khách đổi thẻ/phương thức, sinh key mới trên cùng orderId.

A5 ở bước 6: gửi lại cùng key → trả payment cũ, không trừ kho hay cộng điểm lặp.

A6: khách hủy đơn chưa thanh toán và không có PROCESSING → CANCELLED, hoàn kho/mã, giỏ vẫn có thể sửa.

A7: hai client tranh phần cuối → một transaction thắng, bên còn lại retry rồi 409 nếu hết kho.

2.3.3 Ngoại lệ

E1: JWT hết hạn → 401, yêu cầu đăng nhập lại, không tự xóa giỏ.

E2: cùng key nhưng khác order/method/user → 422 IDEMPOTENCY_KEY_REUSED.

E3: mất mạng sau POST /payments → giữ key, gửi lại chính yêu cầu để lấy kết quả; không tạo đơn mới.

E4: key vẫn PROCESSING sau 5 giây → 409 PAYMENT_IN_PROGRESS; UI cho retry cùng key. Nếu API crash ở khoảng giữa hai transaction, mock hiện chưa có bộ reconciliation tự động; cần xử lý vận hành, không hứa bảo đảm exactly-once qua mọi sự cố.

E5: lỗi 5xx trước khi client nhận POST /orders có thể đã commit: API tạo đơn chưa có create-order idempotency. UI không tự retry POST /orders và hướng dẫn xem lịch sử trước khi tạo lại.

2.3.4 Kịch bản chấp nhận

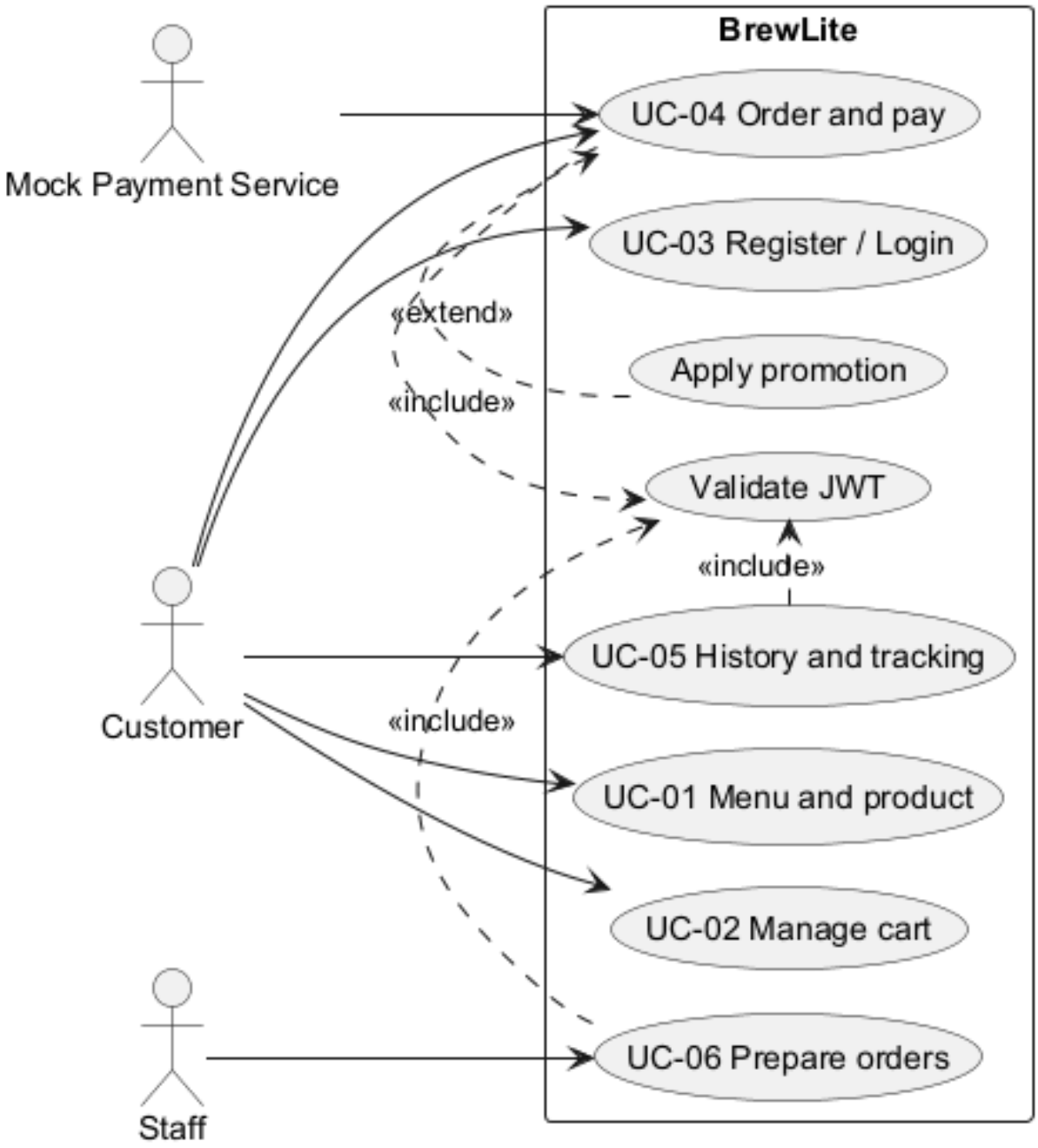
Given giỏ Cappuccino M x1 và Trà đào L + trân châu x2, When WALLET thành công, Then subtotal 158.000, PAID và 15 điểm. Given thẻ 4000000000000002, Then PAYMENT_FAILED và không xóa giỏ. Given hai POST /payments đồng thời cùng key, Then cùng payment id và đúng một charge. Các trường hợp này được kiểm chứng bằng test integration/E2E; xem ma trận truy vết.

Chương 3

Mô hình hóa hệ thống

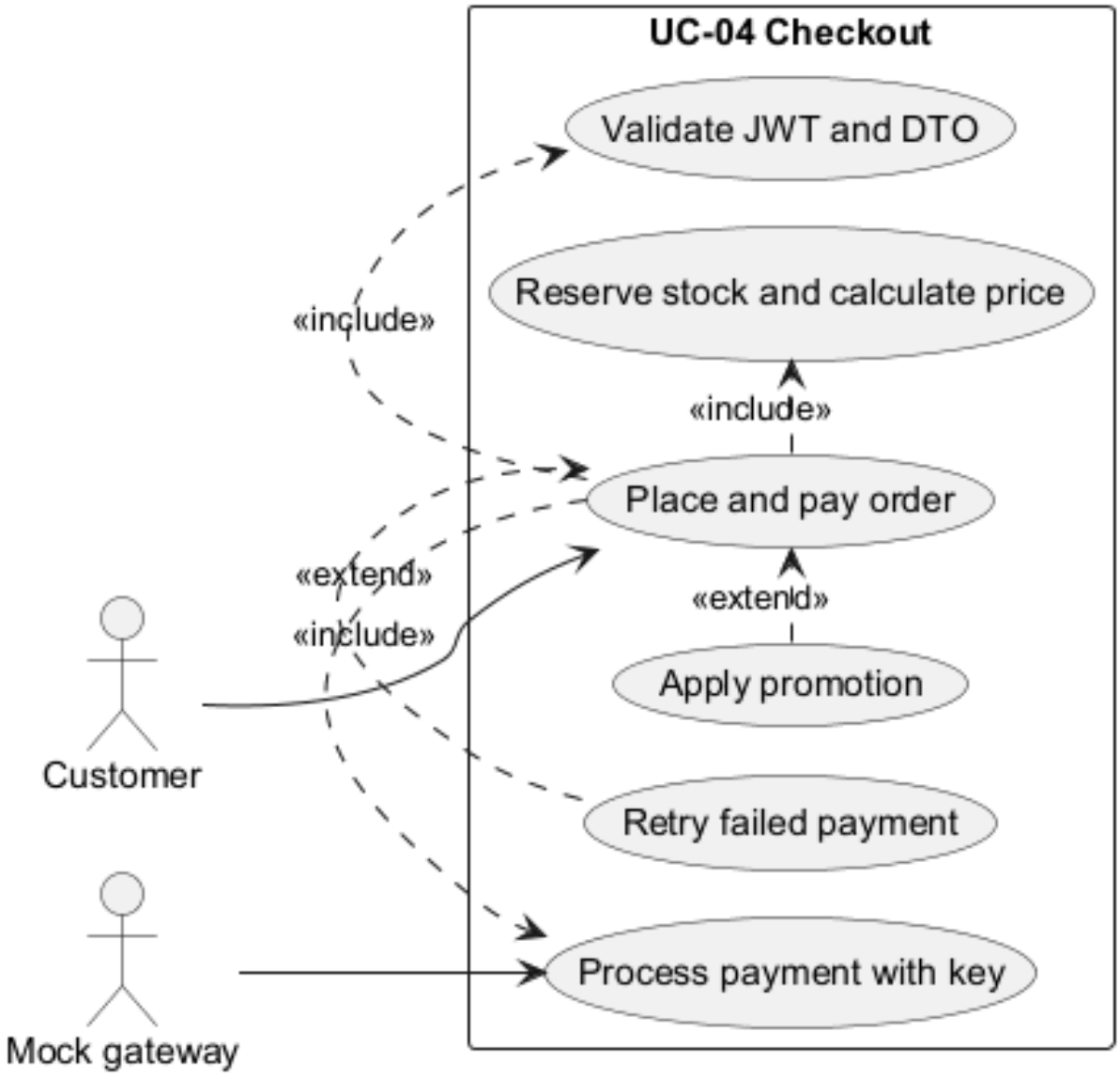
Các sơ đồ sau có nguồn PlantUML và bản SVG/PNG trong docs/diagrams. Sequence mô tả cả transaction tạo đơn và hai transaction thanh toán; state machine phân biệt quyền gateway với thao tác nhân viên. Class diagram thể hiện multiplicity và enum tương ứng Prisma schema.

3.1 Use case tổng quát



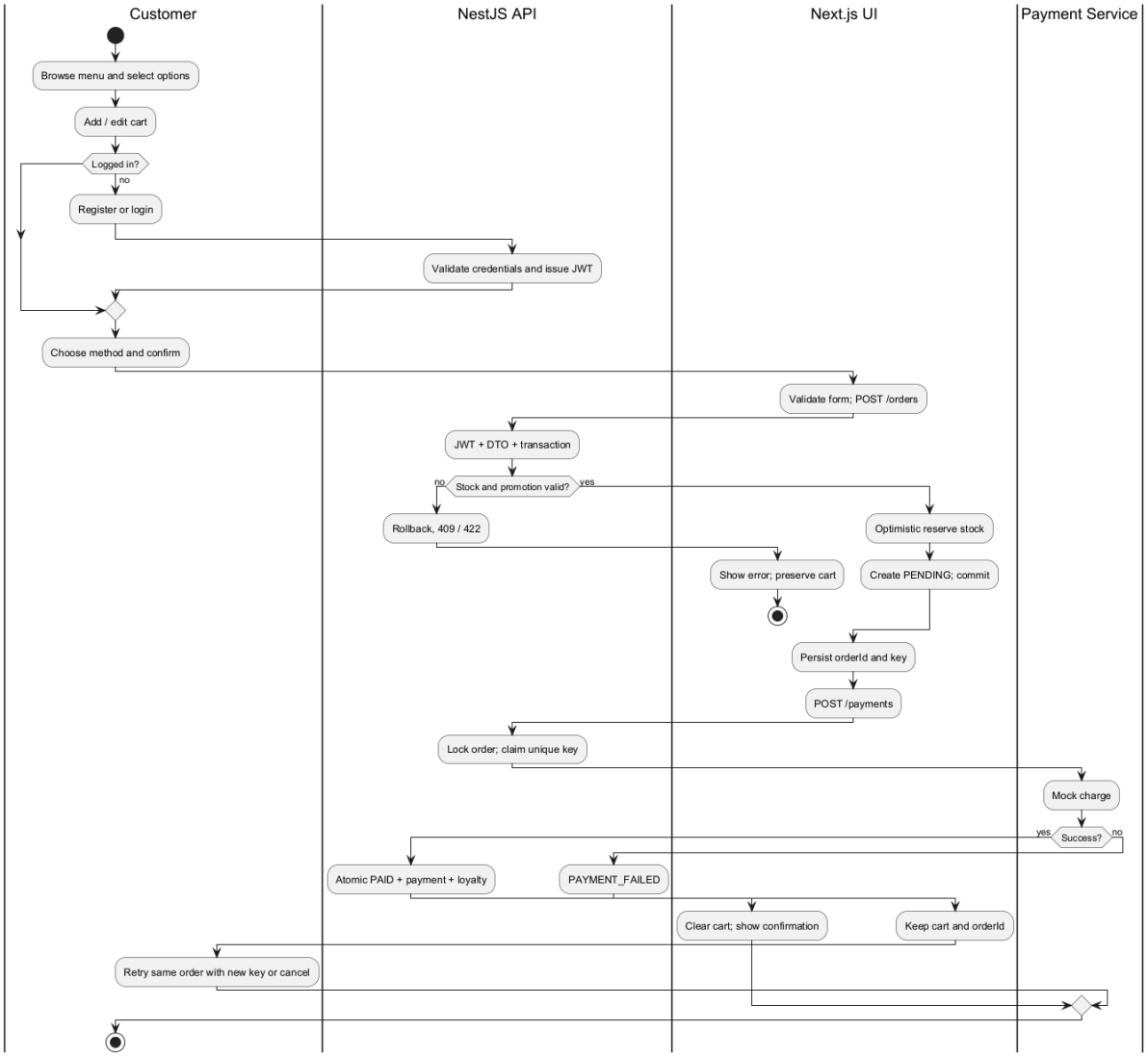
Hình 3.1: Use case tổng quát

3.2 Use case trọng tâm đặt và thanh toán



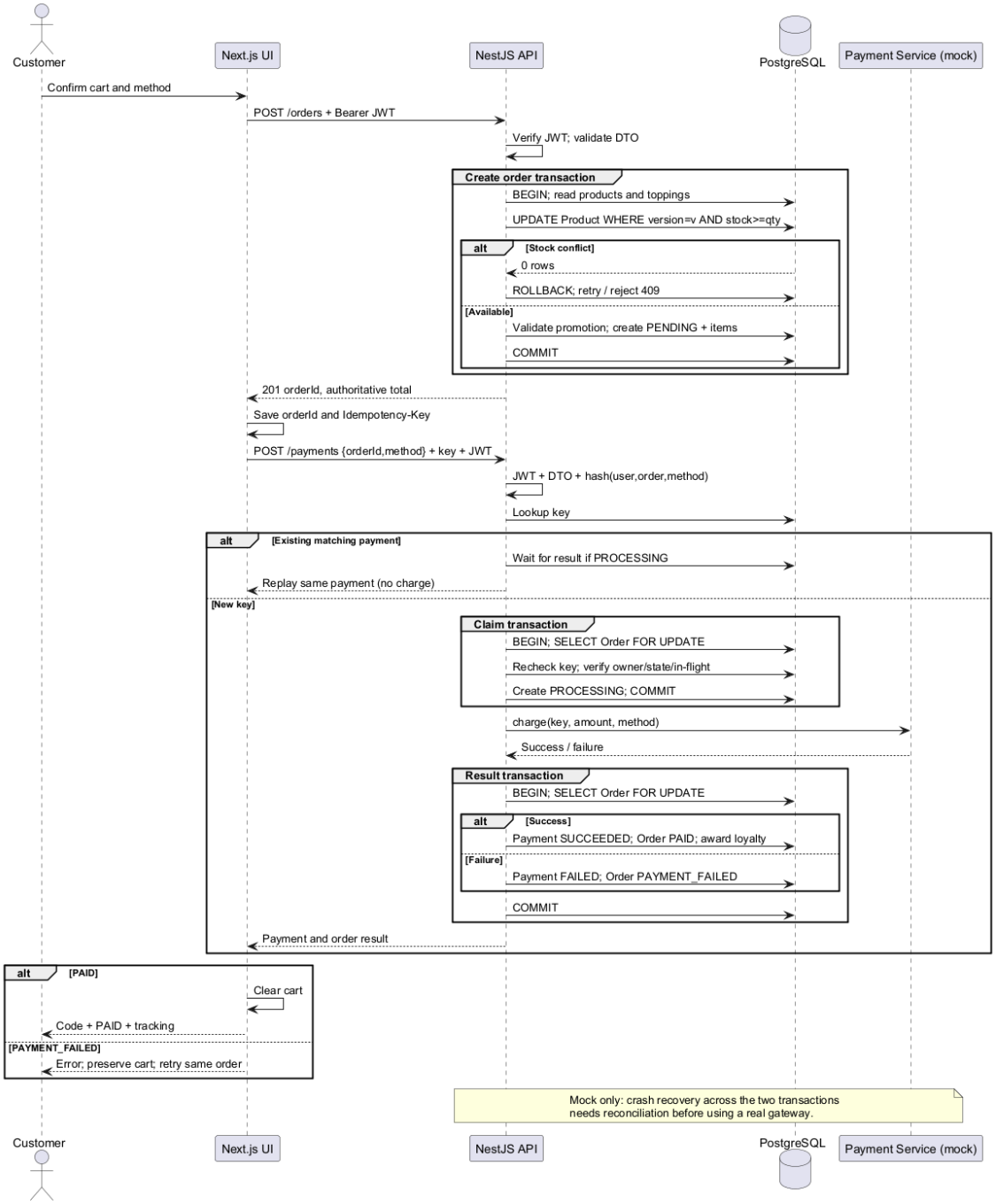
Hình 3.2: Use case trọng tâm đặt và thanh toán

3.3 Activity đặt và thanh toán với swimlane



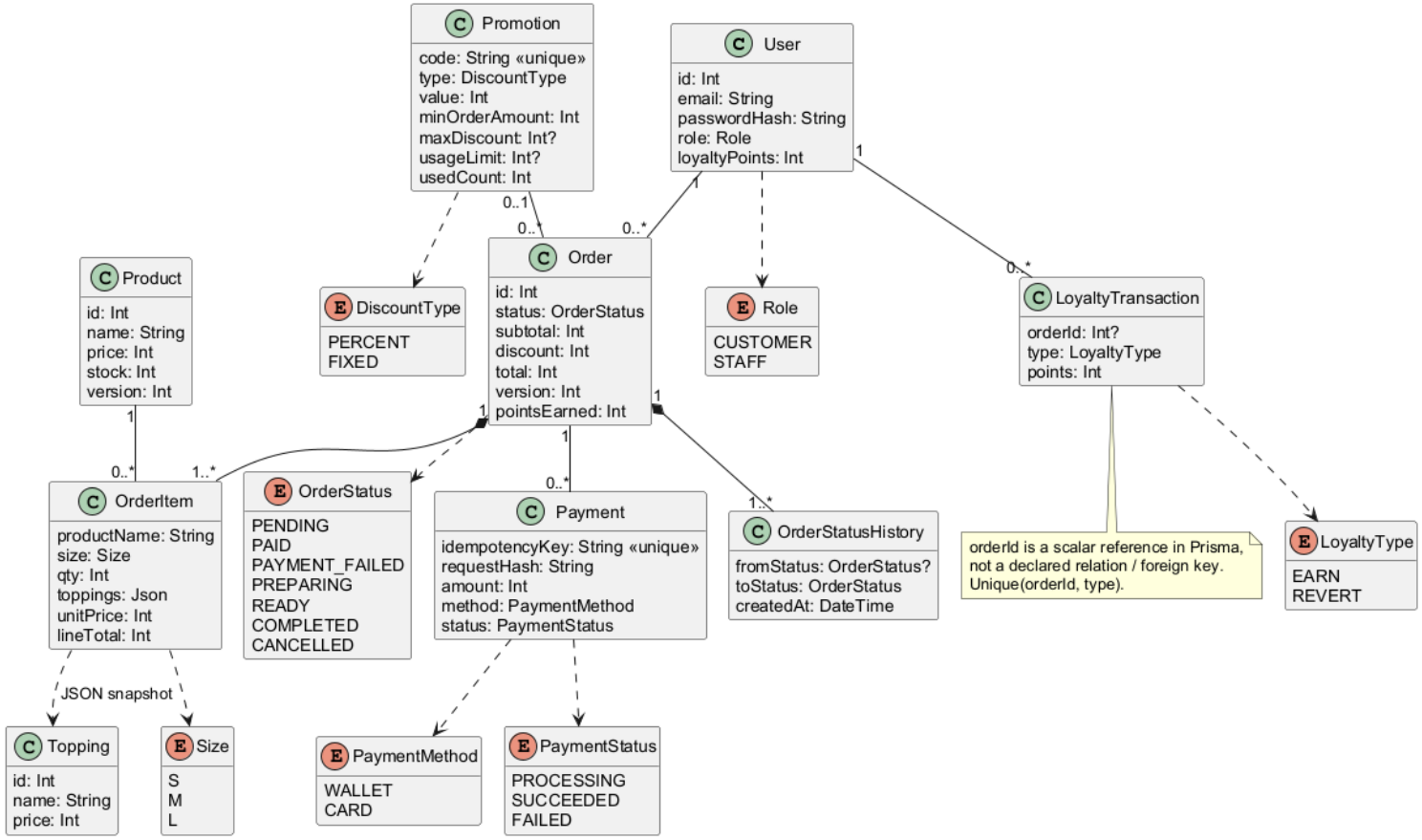
Hình 3.3: Activity đặt và thanh toán với swimlane

3.4 Sequence đặt và thanh toán idempotent



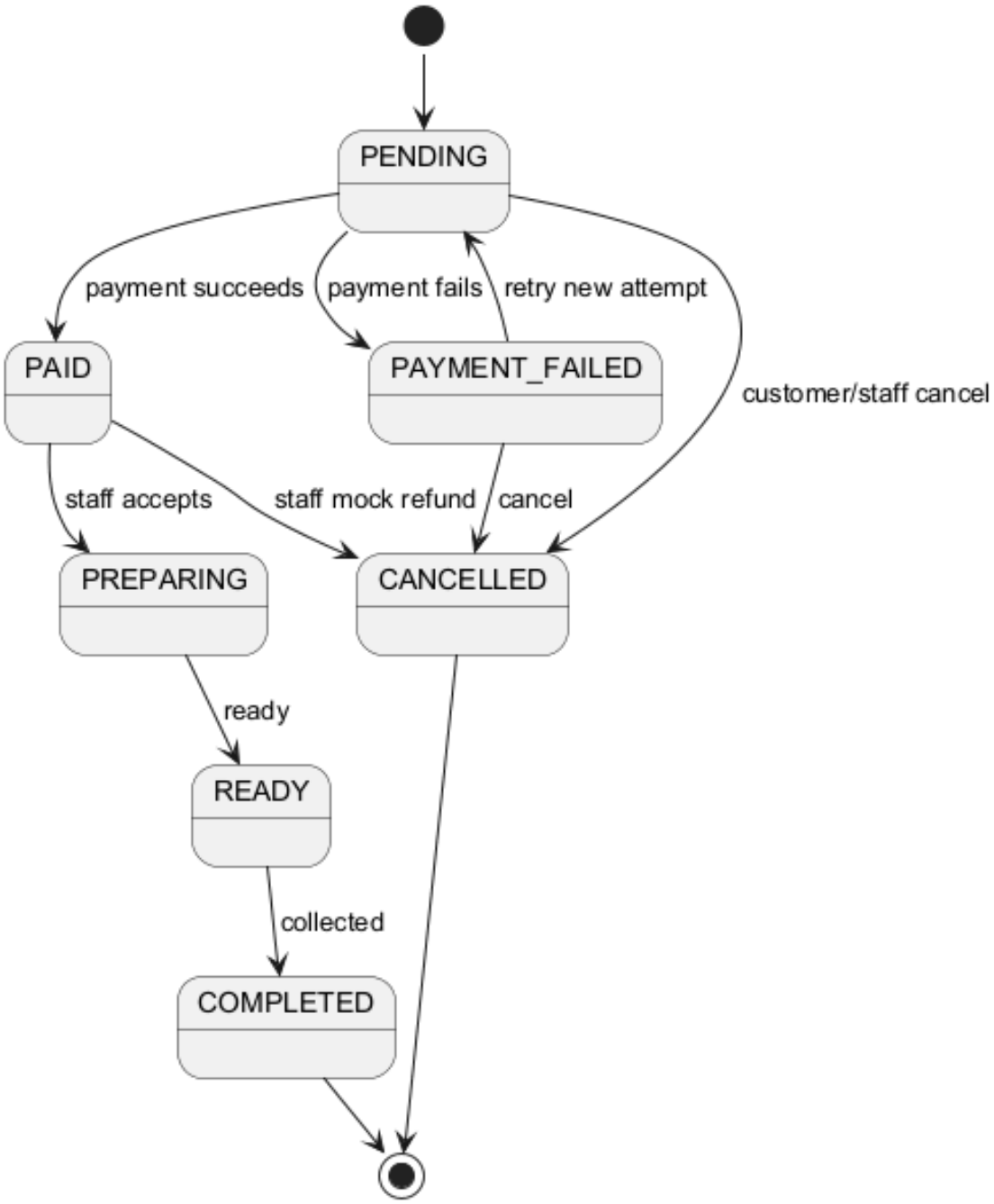
Hình 3.4: Sequence đặt và thanh toán idempotent

3.5 Class diagram domain và enum



Hình 3.5: Class diagram domain và enum

3.6 Máy trạng thái đơn hàng



Hình 3.6: Máy trạng thái đơn hàng

Chương 4

Thiết kế kiến trúc

4.1 Kiến trúc BrewLite

4.1.1 Phương pháp phân lớp và module domain

Browser/PWA → Next.js App Router → route proxy /api → NestJS → Prisma → PostgreSQL. UI không truy cập DB, không tự quyết định giá hay trạng thái thanh toán. Proxy cùng origin giúp trình duyệt không cần biết DNS nội bộ Docker. Gateway thanh toán là module mock trong API, không phải dịch vụ ngân hàng.

Thành phần	Trách nhiệm	Mã nguồn
Next.js	Menu, tùy chọn, cart, auth, checkout, tracking, staff và PWA	apps/web/src/app, components, lib
Proxy	Chuyển tiếp các API được cho phép; timeout, không cache	apps/web/src/app/api/[...path]/route.ts
Auth	DTO, bcrypt, JWT và role guards	apps/api/src/auth
Products	Menu/chi tiết/topping	apps/api/src/products
Orders	Transaction, snapshot giá, optimistic kho, state machine	apps/api/src/orders
Payments	Claim idempotency, khóa Order, mock charge, kết quả atomic	apps/api/src/payments
Promotions / loyalty	Giảm tiền có điều kiện, giới hạn lượt, cộng/ thu hồi điểm	apps/api/src/promotions, loyalty
Persistence	Prisma schema, migration, seed	apps/api/prisma, src/database
Vận hành	Env validation, request ID, health DB, log không chứa body	apps/api/src/config, common, health

4.1.2 Giao dịch và nhất quán

Tạo đơn dùng ReadCommitted + UPDATE có version và stock, retry tối đa 8 lần có backoff. Nhiều dòng cùng sản phẩm được gộp số lượng trước khi giữ kho. Mã khuyến mãi và ghi đơn nằm trong transaction nên lỗi rollback cả kho. Dòng Order được khóa khi claim payment và hủy/cập nhật để tránh race. Unique Payment.idempotencyKey bảo vệ cả khi API nhiều replica. Hash gồm user/order/method, không lưu số thẻ.

Gateway chạy ngoài transaction để không giữ khóa khi chờ mạng. Transaction kết quả lưu payment, trạng thái và điểm cùng lúc. Giới hạn: crash giữa claim và finalize có thể để lại PROCESSING; chưa có outbox/reconciliation. Giải pháp đủ cho mock bài tập, chưa áp dụng cho thu tiền thật. POST /orders chưa có idempotency riêng.

4.1.3 Triển khai

Local: Docker Compose 3 dịch vụ, health dependency, persistent volume. Production được chuẩn bị cho VPS với Caddy HTTPS, web/api nội bộ và DB không publish cổng; GHCR images theo SHA; SSH deploy, migration, smoke và khôi phục image cũ nếu smoke thất bại. Sơ đồ deployment ghi rõ local kiểm chứng và production chưa được cấp host. Không gán tên Render/ Railway khi chưa dùng thực tế.

4.1.4 Trade off

Monolith module giảm chi phí vận hành và cho transaction DB đơn giản; quy mô lớn phải tách gateway/worker và giám sát. Cart persist tăng tiện lợi, nhưng giá là tạm tính; server luôn tính lại. JWT hiện lưu sessionStorage để không kéo dài qua phiên, vẫn cần bảo vệ XSS; hệ thống thực nên dùng cookie HttpOnly và CSRF thích hợp. PWA hỗ trợ cài đặt và màn hình offline, không chấp nhận giao dịch offline.

4.2 ADR 001 Monorepo npm workspaces

Ngày: 2026-11-06. Trạng thái: Accepted (quyết định thiết kế; không phải minh chứng production).

4.2.1 Bối cảnh và quyết định

Đồng bộ frontend/backend/test và một lockfile; dễ chạy CI và Compose. Đổi lại Docker build context lớn; dùng .dockerignore. Không chọn microservices vì nghiệp vụ và nhóm nhỏ.

4.2.2 Hệ quả

Được truy vết trong architecture.md, schema/mã nguồn và sơ đồ; thay đổi quyết định phải bổ sung ADR thay thế thay vì sửa lịch sử.

4.3 ADR 002 PostgreSQL và Prisma

Ngày: 2026-11-06. Trạng thái: Accepted (quyết định thiết kế; không phải minh chứng production).

4.3.1 Bối cảnh và quyết định

Migration có version, snapshot giá, transaction và optimistic version stock. Không dùng SQLite vì cần chứng minh concurrency PostgreSQL đúng môi trường đề bài.

4.3.2 Hệ quả

Được truy vết trong architecture.md, schema/mã nguồn và sơ đồ; thay đổi quyết định phải bổ sung ADR thay thế thay vì sửa lịch sử.

4.4 ADR 003 Proxy cùng origin

Ngày: 2026-11-06. Trạng thái: Accepted (quyết định thiết kế; không phải minh chứng production).

4.4.1 Bối cảnh và quyết định

UI gọi /api; Next route chuyển tới API_INTERNAL_URL ở runtime. Chỉ cho prefix API hợp lệ và không chuyển cookie người dùng; JWT được gửi Authorization. Thêm một hop để đổi lấy cấu hình Docker đơn giản.

4.4.2 Hệ quả

Được truy vết trong architecture.md, schema/mã nguồn và sơ đồ; thay đổi quyết định phải bổ sung ADR thay thế thay vì sửa lịch sử.

4.5 ADR 004 Thanh toán mock idempotent

Ngày: 2026-11-06. Trạng thái: Accepted (quyết định thiết kế; không phải minh chứng production).

4.5.1 Bối cảnh và quyết định

Unique key toàn DB, hash user/order/method, khóa dòng Order và hai transaction bao quanh gateway. Replay trả cùng payment; lỗi payload trả 422. Sự cố crash cần reconciliation trước cổng thật; không tuyên bố exactly-once xuyên sự cố.

4.5.2 Hệ quả

Được truy vết trong architecture.md, schema/mã nguồn và sơ đồ; thay đổi quyết định phải bổ sung ADR thay thế thay vì sửa lịch sử.

4.6 ADR 005 Delivery bằng image bất biến

Ngày: 2026-11-06. Trạng thái: Accepted (quyết định thiết kế; không phải minh chứng production).

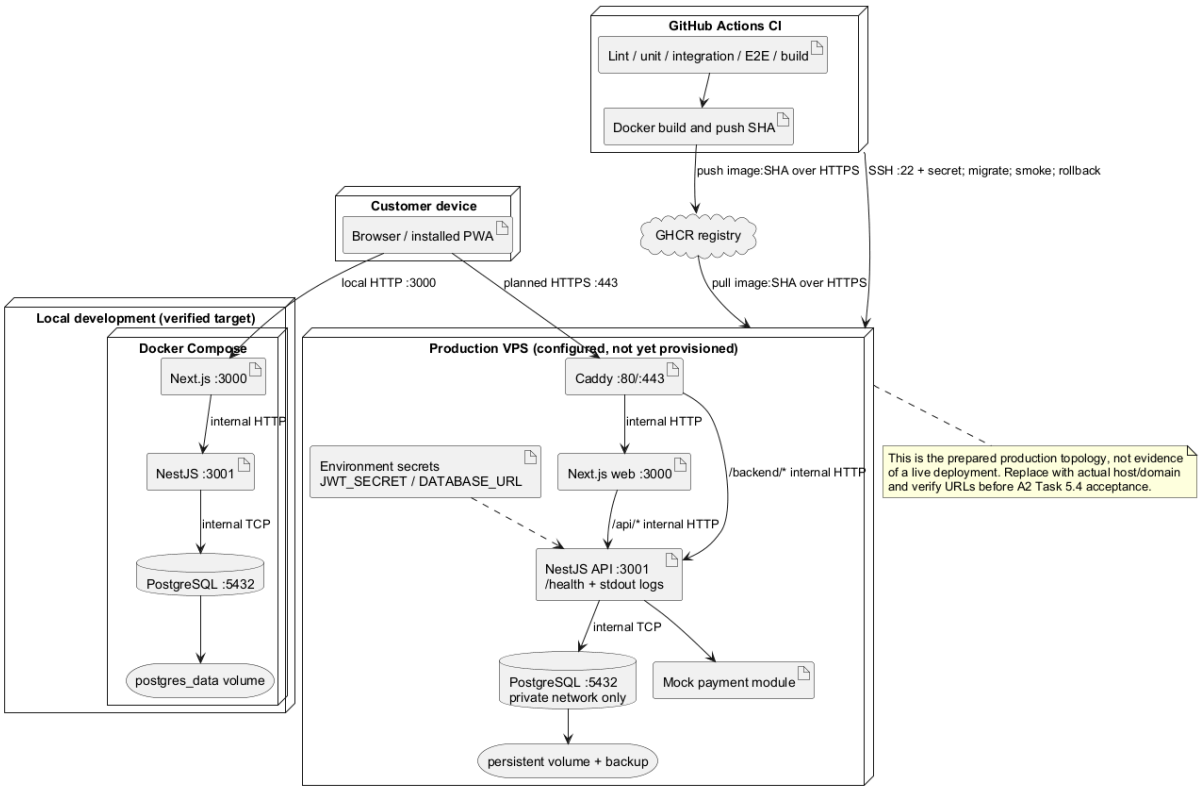
4.6.1 Bối cảnh và quyết định

GHCR tag SHA, VPS Compose + Caddy, migration trước service, smoke sau deploy, rollback image trước đó. Không hạ migration tự động; yêu cầu migration tương thích ngược và backup trước thay đổi phá hủy.

4.6.2 Hệ quả

Được truy vết trong architecture.md, schema/mã nguồn và sơ đồ; thay đổi quyết định phải bổ sung ADR thay thế thay vì sửa lịch sử.

4.7 Deployment diagram



Hình 4.1: Local và CI cùng topology VPS production được chuẩn bị. Chưa gắn host production thật.

Chương 5

Quản lý Agile Scrum

5.1 Product Backlog

Product vision: khách chọn đồ uống, thanh toán không tiền mặt và nhận nhanh ở quầy. Owner được mô tả theo vai trò; tên thành viên thật phải bổ sung khi nhóm cung cấp. Điểm là ước lượng kế hoạch, không phải số điểm chấm được bảo đảm.

ID	User story và giá trị	Ưu tiên	SP	A1 Task	A2 Sprint	Acceptance
US-01	Là developer, tôi muốn project có cấu trúc để chạy và bàn giao	Must	2	1	1	Next/Nest, README, env mẫu, Git
US-02	Là khách, tôi muốn đọc menu chính xác từ API	Must	2	2	1	GET /products, seed, API test
US-03	Là khách, tôi muốn menu dùng được trên điện thoại	Must	3	3	1	Loading/empty/error và lọc/ tìm
US-04	Là khách, tôi muốn chọn size/ topping và biết giá	Must	3	4	1	Chi tiết, lựa chọn và tính giá
US-05	Là khách, tôi muốn chỉnh giỏ trước khi mua	Must	3	5	2	Qty/xóa, badge/tổng, persist
US-06	Là khách, tôi muốn đơn lưu đúng trong hệ thống	Must	5	6	2	DTO, JWT, PENDING, transaction
US-07	Là khách, tôi muốn tài khoản riêng để bảo vệ đơn	Must	3	7	2	Bcrypt, register/login JWT, ownership
US-08	Là khách, tôi muốn thanh toán Ví/ Thẻ mô phỏng an toàn	Must	5	8	2	PAID/FAILED, key, giữ giỏ khi lỗi
US-09	Là khách/giảng viên, tôi muốn xác nhận và tái tạo bản chạy	Must	3	9	2	History, Compose, demo và README
US-10	Là PO, tôi muốn nghiệp vụ không sai khi gửi lặp/đồng thời	Must	8	10	2	State, idempotency, không âm kho, promo/loyalty

5.1.1 Definition of Ready

Story có giá trị và acceptance kiểm chứng được, FR/UC liên quan, dependency rõ, ước lượng và phạm vi đủ nhỏ để demo. Task 2/3 của A2 dựa trên SRS/use case đã lưu trước khi hiện thực.

5.1.2 Definition of Done

Build/lint/typecheck đạt; acceptance đã kiểm chứng; API validate và UI xử lý loading/error; có test tương ứng cho rủi ro; không có secret; migration/README đồng bộ; source diagram và ảnh đồng bộ; PR được review và CI xanh; tài liệu ghi đúng bằng chứng. Chỉ đánh dấu production Done sau khi HTTPS/frontend/API/health hoạt động.

5.1.3 Vai trò Scrum

PO: xác nhận phạm vi và nghiệm thu. SM: theo dõi blocker, điều phối event. Developers: phân tích, code, test, docs, vận hành. Tên thành viên đảm nhận từng vai trò được ghi ở trang bìa báo cáo.

5.2 Kế hoạch Sprint và ánh xạ hai Assignment

Assignment 1 gợi ý 3 Sprint: A1-S1 task 1–4; A1-S2 task 5–7; A1-S3 task 8–10. Assignment 2 quy định 2 Sprint hiện thực sau phân tích/thiết kế: A2-S1 tập trung nền tảng/menu; A2-S2 hoàn thành end-to-end và delivery. Dùng cùng Product Backlog, ánh xạ A1-S2 và S3 vào A2-S2; không tuyên bố đề A2 yêu cầu 3 Sprint.

Sprint A2	Goal	Backlog	Increment / bằng chứng nghiệm thu
Sprint 1	Nền tảng tái lập và xem menu thật	US-01..04; requirement/ model/ architecture; CI cơ bản	API menu/chi tiết, giao diện mobile, seed, test API/component, SRS + UML + ADR
Sprint 2	Đặt đến xác nhận và bàn giao	US-05..10; integration/ E2E; Docker, CD, báo cáo	Auth/giỏ/đơn/thanh toán/lịch sử/staff; concurrency tests; image pipeline; deploy nếu có host

5.2.1 Sprint Planning

Ưu tiên Must và các phụ thuộc: docs yêu cầu → mô hình → kiến trúc → API menu/UI → auth/ order/cart → payment → test/delivery. Mỗi story tách thiết kế/code/test/docs; ước lượng tổng

37 SP. Đây là kế hoạch phân rã để nhóm sử dụng; không phải timesheet nhiều tuần đã diễn ra.

5.2.2 Daily Scrum

Nhóm cập nhật ba mục: tiến độ so với Sprint Goal, kế hoạch tiếp theo và blocker; giới hạn 15 phút. Các blocker kỹ thuật ghi nhận trong Sprint 2: cần database test tách biệt cho integration test, CI phát hiện lỗi kiểu trong component test, bổ sung khóa chống race giữa thanh toán và hủy đơn. Mỗi blocker được xử lý bằng commit/PR có liên kết trong lịch sử Git.

5.2.3 Burndown

`burndown-plan.csv` chỉ biểu diễn đường kế hoạch 10 ngày cho 37 SP. Đường Actual được cập nhật từ issue board tại mỗi Daily Scrum. Nhóm cần cập nhật từ issue board tại Daily Scrum trước nghiệm thu quy trình. Lịch sử Git và Actions phản ánh công việc kỹ thuật thực hiện trong phiên này.

5.3 Sprint Review và Retrospective

5.3.1 Review kỹ thuật ngày 06 tháng 11 năm 2026

Increment được chuẩn bị cho demo: menu → size/topping → giỏ → register/login → tạo đơn → Ví/Thẻ mock → xác nhận/lịch sử. Phần mở rộng: quầy pha chế, khuyến mãi, điểm, hủy và hoàn kho. Kết quả test/CI được liên kết ở `../quality/verification.md`; không đồng nhất review tự động với nghiệm thu chính thức của PO/giảng viên.

Các tình huống review: giỏ giữ qua reload; thẻ từ chối giữ giỏ; retry cùng Order; 10 payment cùng key; 20 khách tranh 5 phần; STAFF không tự đặt PAID và khách không đổi state. Bằng chứng nằm ở test và Actions, thay vì lời khẳng định không kèm kiểm chứng.

5.3.2 Retrospective từ công việc đã diễn ra

Điều hiệu quả: một schema dùng cho API/test/diagrams; tái sử dụng backend Sprint 1 làm nền; tách database test `brewlite_test` để integration test không ghi đè dữ liệu phát triển; CI tìm ra lỗi kiểu mà unit runner transpile không phát hiện.

Điều cần cải thiện: viết DTO/API contract trước UI; tính trước giới hạn transaction pool dưới tải; chạy typecheck cùng test; quản lý phiên thanh toán bền hơn; cấp môi trường production sớm.

Hành động tiếp theo: chuyển JWT sang cookie HttpOnly khi triển khai ứng dụng thực; thêm create-order idempotency, thời hạn giữ kho và reconciliation PROCESSING; rà soát dependency

audit; kiểm tra backup phục hồi; nhóm điền vai trò thực và tổ chức nghiệm thu với giảng viên.

5.3.3 Phản hồi khách hàng

Phản hồi của giảng viên/PO được ghi sau buổi Sprint Review cuối. Khi demo, ghi người tham dự, ngày, acceptance đạt/chưa đạt và chuyển góp ý thành issue có ưu tiên.

Chương 6

Hiện thực phần mềm

6.1 Frontend Next.js

Menu gọi API thật và hiển thị các trạng thái loading, empty và error. Khách lọc danh mục, tìm đồ uống, chọn size S/M/L và topping. Zustand persist giữ giỏ trên thiết bị, mỗi tổ hợp sản phẩm/size/topping là một dòng; số lượng nguyên trong phạm vi 1–20. Giao diện dùng nhãn form, thông báo lỗi và bố cục responsive. Manifest và service worker hỗ trợ cài PWA và trang offline; không cache giao dịch hoặc API chứa dữ liệu cá nhân.

6.2 Xác thực và phân quyền

DTO chuẩn hóa email, kiểm tra độ dài và định dạng. Mật khẩu băm bcrypt; JWT có subject và role. API kiểm tra quyền sở hữu đơn, trả 404 khi khách truy cập đơn người khác; chỉ STAFF cập nhật trạng thái. Token lưu ở sessionStorage cho phiên demo; cookie HttpOnly và cơ chế chống XSS/CSRF đầy đủ là hướng hoàn thiện trước vận hành thực.

6.3 Tạo đơn và optimistic locking

Server lấy lại giá và topping từ DB, gộp số lượng theo productId và cập nhật tồn kho theo thứ tự id. Câu lệnh update chỉ thành công khi version khớp và stock đủ. Nếu count bằng 0, toàn transaction rollback và retry có backoff, tối đa 8 lần. Tạo đơn, snapshot items, giữ kho và sử dụng promotion nằm cùng transaction.

```
UPDATE Product SET stock = stock - qty, version = version + 1
WHERE id = productId AND version = expectedVersion AND stock >= qty;
```

6.4 Thanh toán idempotent

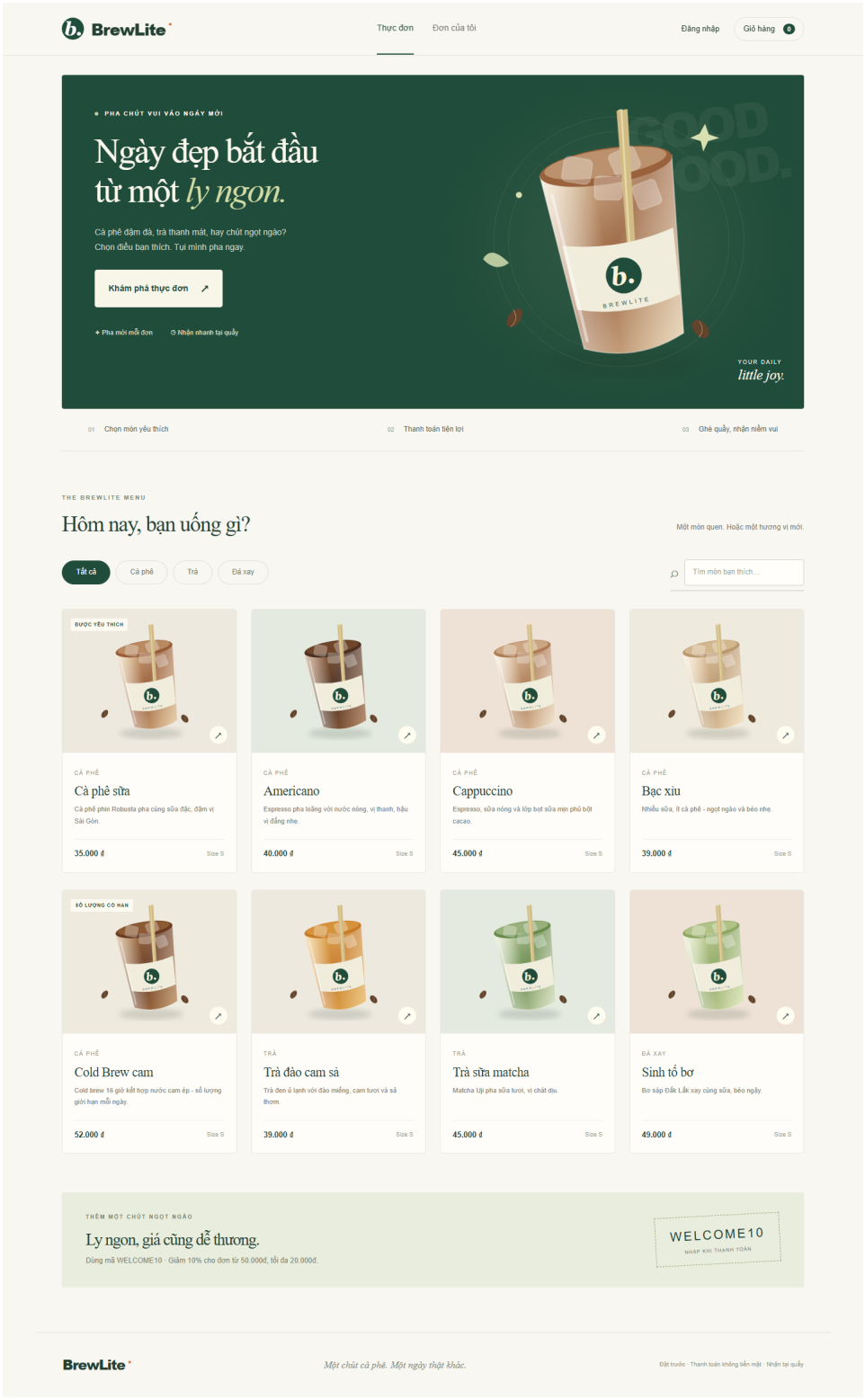
Payment có unique Idempotency-Key, hash của user/order/method và số tiền lấy từ đơn. Khi claim, API khóa dòng Order và đọc lại key sau khóa để request đồng thời không bị xem là một payment khác. Gọi gateway ngoài transaction, sau đó finalize payment, state và loyalty nguyên tử; retry finalize dùng kết quả đã nhận, không charge lại. Replay cùng key trả cùng payment; dùng key cho payload khác bị từ chối 422. Mỗi lần thử mới sau FAILED sử dụng key mới trên cùng đơn.

6.5 Hủy và chuyển trạng thái

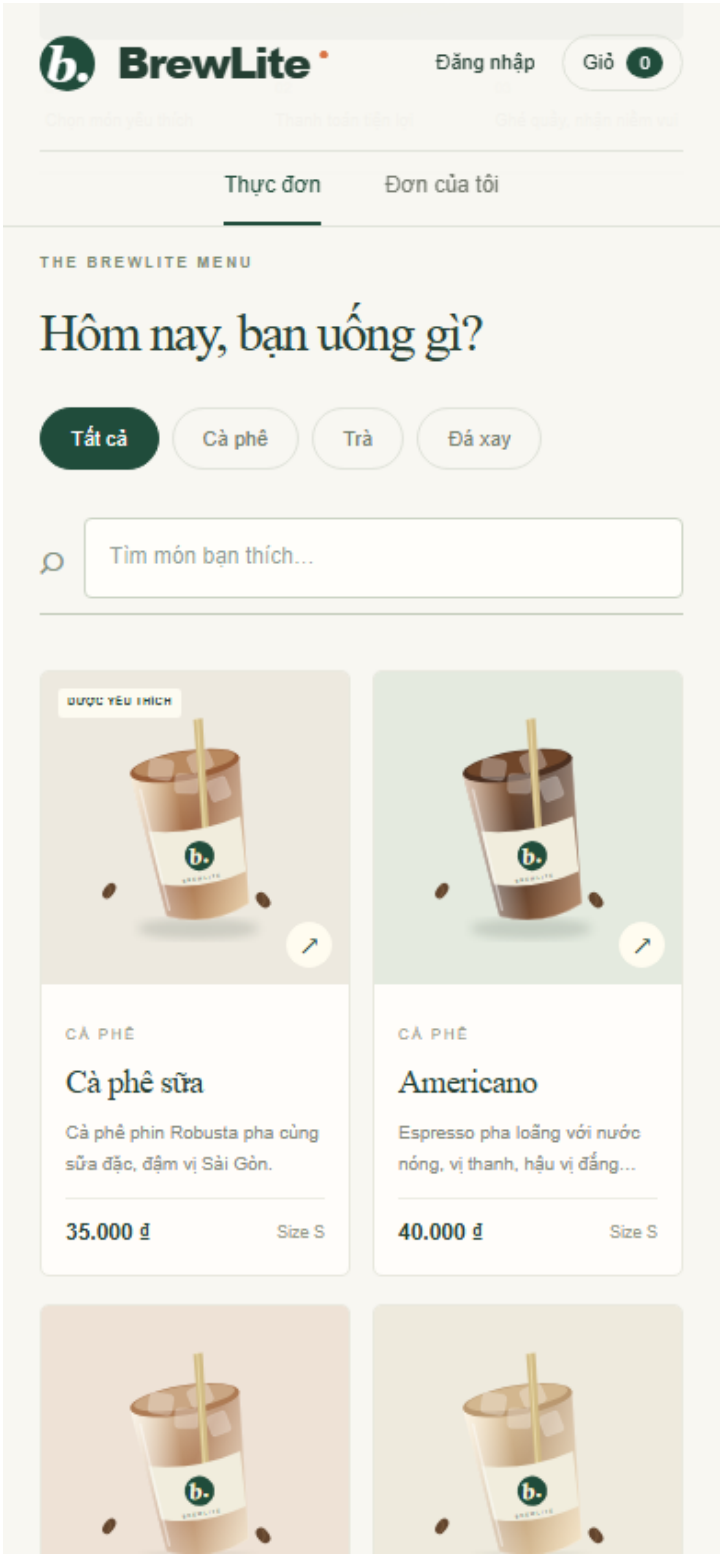
Các thao tác hủy/thay đổi state khóa cùng dòng Order. Không hủy trong lúc payment PROCESSING. State machine chặn chuyển sai; hủy hợp lệ hoàn kho, trả lượt promotion và thu hồi loyalty khi PAID bị hủy tại quầy. Đây là hoàn tiền mô phỏng, không phải refund ngân hàng.

6.6 Khôi phục checkout

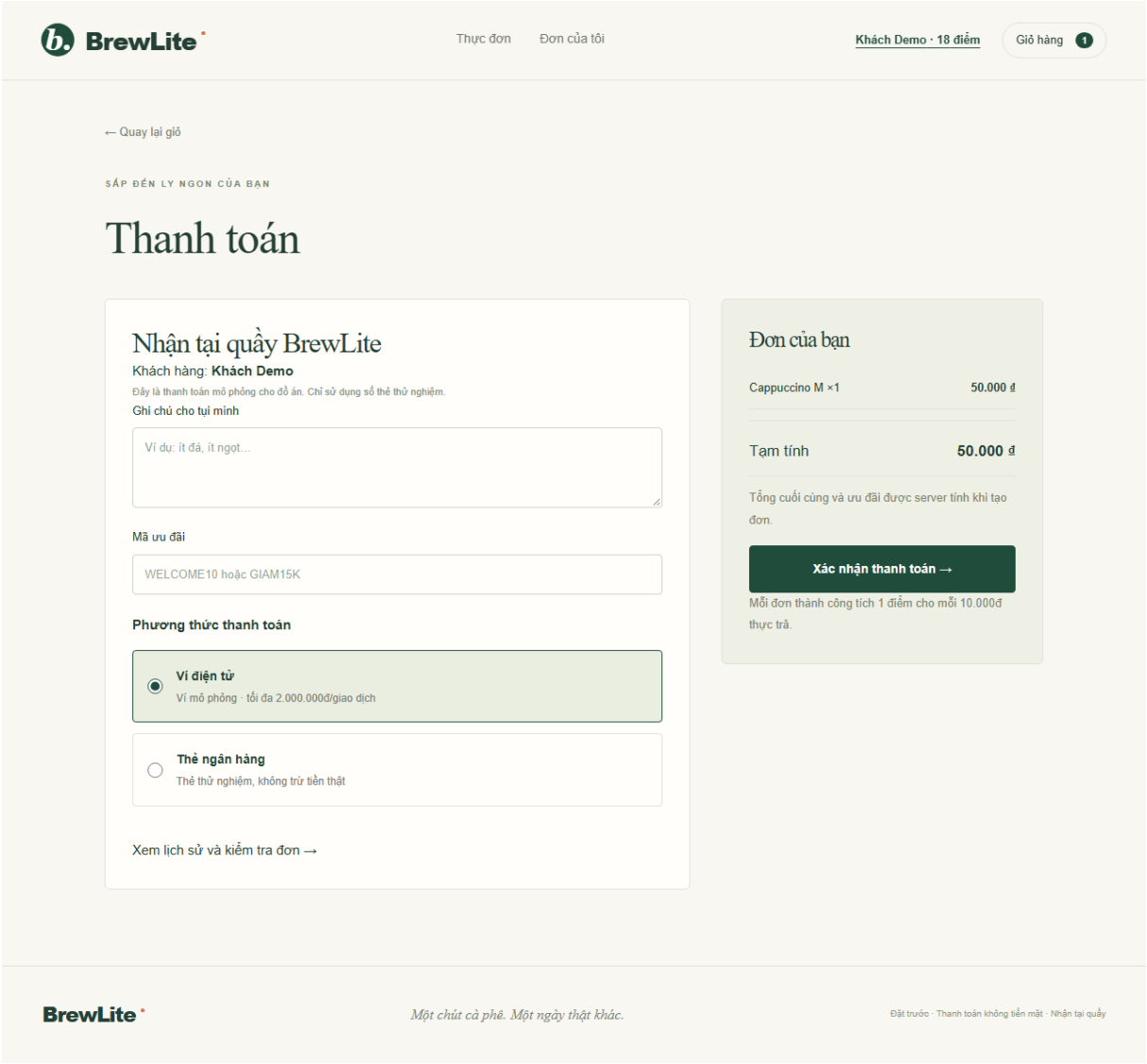
UI giữ orderId và key trong sessionStorage khi chưa rõ kết quả thanh toán, giữ giỏ nếu thất bại và thử lại trên cùng order. Pending được xóa phù hợp khi đổi tài khoản; hủy một đơn khác không được xóa nhầm pending. Nếu kết quả tạo đơn không rõ, UI yêu cầu xem lịch sử và không tự động gửi POST tạo đơn lần nữa. API tạo đơn chưa có idempotency riêng; process crash giữa claim và finalize cần reconciliation trước khi tích hợp cổng thật.



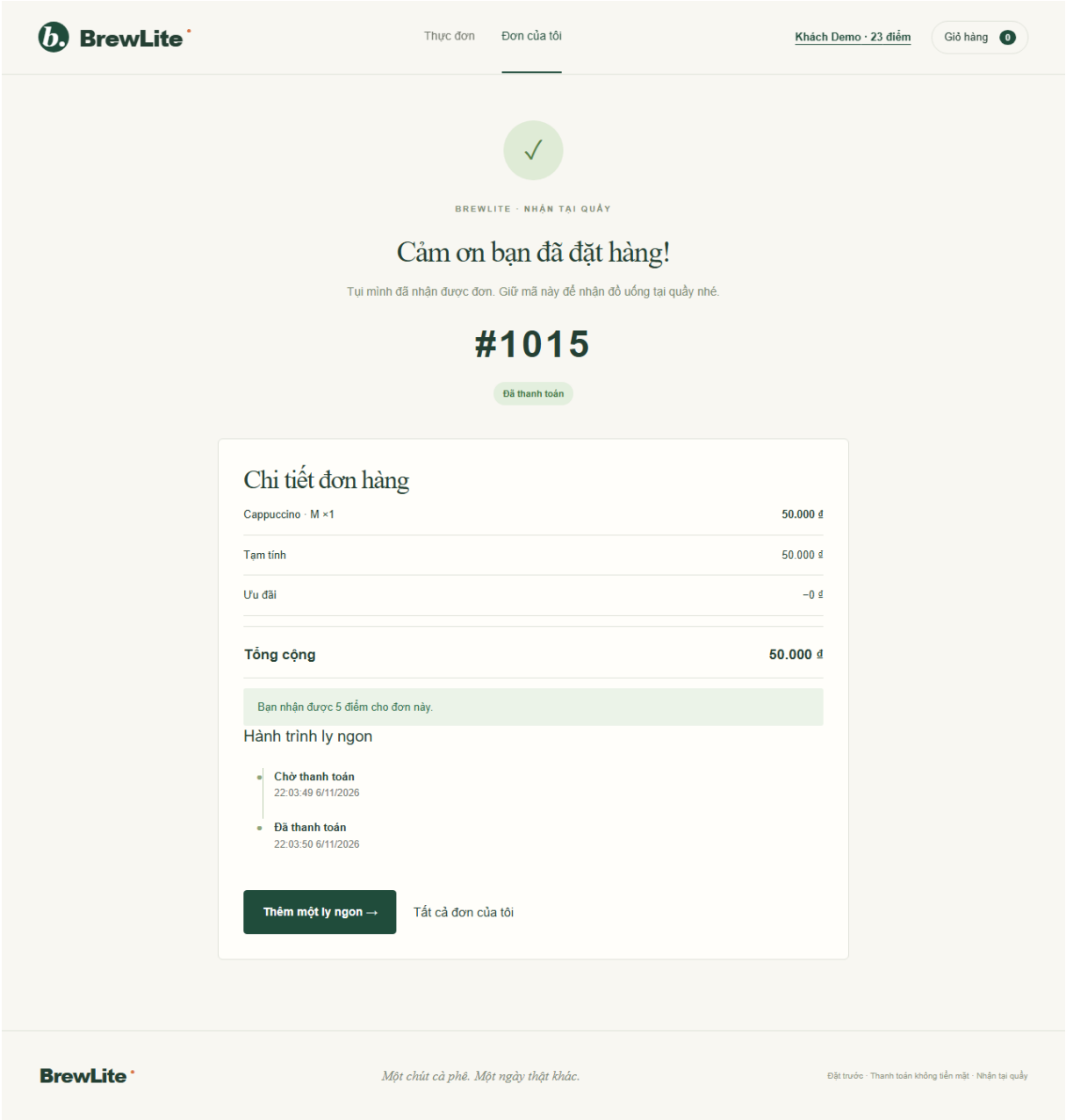
Hình 6.1: Giao diện thực đơn trên desktop



Hình 6.2: Giao diện thực đơn trên điện thoại



Hình 6.3: Giao diện thanh toán mô phỏng



Hình 6.4: Xác nhận đơn hàng thực tế

Chương 7

Kiểm thử và bằng chứng

7.1 Bằng chứng kiểm chứng

Ngày kiểm tra: 06/11/2026, Windows 11, Node 24.16.0 local, Node 22 trong Docker/CI, PostgreSQL 17 Docker. Payment/order/checkout/cart đã qua rà soát code (xem `code-review.md`); các lỗi pending và số lượng phát hiện khi rà soát đã được sửa và có test. Giới hạn crash recovery của mock được ghi trong SRS/ADR, không tự gọi lại gateway sau crash vì cache mock chỉ ở RAM.

Kiểm tra	Kết quả quan sát
Unit backend	50 tests / 6 suites pass
Unit/component web	8 tests / 2 files pass (cart + menu component)
Integration PostgreSQL	31 tests / 4 suites pass; database riêng brewlite_test
Đồng thời kho	20 requests, stock 5: đúng 5 thành công; thêm qty=2 không âm kho
Đồng thời thanh toán	10 cùng key: cùng payment, đúng một charge; hai key/đơn không charge hai lần
Docker	Web/API/DB build và cả ba healthy trên localhost
Typecheck/lint/build	Đạt trên local và Docker build; GitHub CI chạy độc lập
E2E desktop/mobile	6 tests pass: 3 tình huống × desktop và mobile; success, decline/retry, resume/cancel pending
HTTPS production	Chưa có host/domain và deploy secrets; chưa thực hiện

7.1.1 CI và pull request thật

Repository nộp bài Brew_Lite_2 (07/11/2026)

- Repository: https://github.com/Kenkra9999/Brew_Lite_2 — nhận nguyên lịch sử commit, tag `v1.0.0` và 15 issue backlog (milestone Sprint 1/Sprint 2) từ repository làm việc ban đầu.

- Push `main`: https://github.com/Kenkra9999/Brew_Lite_2/actions/runs/42370367806 — Quality Gate xanh toàn bộ (lint, typecheck, unit, integration PostgreSQL, build, E2E desktop/mobile).
- Push tag `v1.0.0`: https://github.com/Kenkra9999/Brew_Lite_2/actions/runs/42370368368 — Delivery: quality xanh, build và push hai image `ghcr.io/kenkra9999/brew_lite_2-api` và `-web` gắn tag commit SHA; job deploy được skip đúng thiết kế vì chưa có host production.
- PR kiểm chứng cổng chất lượng: https://github.com/Kenkra9999/Brew_Lite_2/pull/16 — cố ý thêm `console.log` vi phạm `no-console`. Run https://github.com/Kenkra9999/Brew_Lite_2/actions/runs/42370607079 dừng ở `npm run lint`, typecheck/test/build/E2E bị skip; GitHub báo merge state `BLOCKED`. PR đóng, không merge. Đây là lỗi cố ý để kiểm tra gate, ghi rõ trong mô tả PR.
- PR chuyển repository và cập nhật tài liệu: https://github.com/Kenkra9999/Brew_Lite_2/pull/17 — Quality Gate xanh rồi mới merge (squash, linear history).
- Branch protection `main` bật ngày 07/11/2026: required check `quality`, strict up-to-date, enforce admins, linear history, cấm force push/xóa nhánh.

Repository làm việc ban đầu brewlite2 (06/11/2026)

- Repository: https://github.com/Kenkra9999/brewlite2
- PR: https://github.com/Kenkra9999/brewlite2/pull/1
- Run phát hiện lỗi: https://github.com/Kenkra9999/brewlite2/actions/runs/42236429957 — typecheck bắt `exact` không thuộc ByRoleOptions trong component test. Pipeline dừng, build/E2E không chạy tiếp. Lỗi được sửa ở commit tiếp theo của cùng PR.
- Run thành công: https://github.com/Kenkra9999/brewlite2/actions/runs/42238619259 — lint, typecheck, 50 backend unit, 8 web unit/component, 31 integration, build và 6 E2E desktop/mobile đều xanh.
- GitHub main đã bật required check `quality`, strict up-to-date, enforce admins và linear history ngày 06/11/2026. PR phải xanh mới merge.

7.1.2 Giới hạn và bước kiểm tra bổ sung

Thanh toán chỉ mô phỏng; chưa có reconciliation cho PROCESSING sau process crash; POST /orders chưa idempotent; đơn treo chưa có TTL. JWT ở sessionStorage, chưa có HttpOnly cookie. Dependency audit còn advisory ở bộ phiên bản kế thừa; cần nâng dependency có kiểm thử trước vận hành thật. Các test tự động không chứng minh SLA, không thay nghiệm thu khách hàng hoặc pentest. Chưa tuyên bố đáp ứng production khi mới có Compose và workflow.

7.2 Đo hiệu năng local

Đo 100 request GET /products tuần tự sau 5 lần warmup: p50 4.02 ms, p95 5.92 ms, lớn nhất 7.29 ms. Ngưỡng p95 của bài là 500 ms. Thời điểm đo 2026-11-06T14:53:07.086Z; Node v24.16.0, win32. Đây là phép đo local trên seed nhỏ, không phải cam kết SLA production.

Chương 8

Triển khai và bàn giao

8.1 Hướng dẫn triển khai và rollback

8.1.1 Trạng thái và sơ đồ

Local được chạy qua Compose để kiểm chứng. Production được chuẩn bị cho VPS Linux + Docker + Caddy nhưng chưa có host/domain được cung cấp. `diagrams/deployment.puml` tách local/CI/production và ghi rõ trạng thái này. URL thật và ngày smoke phải được bổ sung sau khi provision. Không dùng URL ngẫu nhiên hoặc giả lập minh chứng deploy.

8.1.2 Chuẩn bị VPS

1. Cấp VPS có Docker Engine/Compose v2, DNS A/AAAA domain trỏ đúng host và mở 80/443. SSH 22 chỉ cho người vận hành/CI; không mở 5432/3001.

2. Tạo `/opt/brewlite/deploy`, copy compose production và Caddyfile. Copy `.env.production.example` thành `/opt/brewlite/.env`, chmod 600 và điền DOMAIN, IMAGE_PREFIX, POSTGRES_PASSWORD, DATABASE_URL, JWT_SECRET. Password trong URL cần URL encode; ví dụ chỉ dùng ký tự URL-safe.

3. Dùng image GHCR có tag full SHA. Package private cần login GHCR trên VPS bằng token chỉ read:packages; không lưu token vào repo/log.

4. Tạo GitHub Environment `production`; secrets `DEPLOY_HOST`, `DEPLOY_USER`, `DEPLOY_SSH_KEY`, `DEPLOY_KNOWN_HOSTS`. Known hosts được xác minh từ chủ VPS; không tắt StrictHostKeyChecking. Tài khoản deploy có quyền Docker (tương đương root trên host), cần máy riêng cho bài.

5. Workflow Delivery chạy quality gate → build images → push GHCR → SSH deploy → migrate → up –wait → HTTPS smoke. Chạy thủ công deploy=false để chỉ xuất image,

deploy=true khi host đã cấu hình. Tag `v*` xuất image; tự deploy khi repository variable `PRODUCTION_ENABLED=true`. Manual deploy=true luôn yêu cầu secrets. Khi chưa có host, job deploy được skip rõ ràng, không coi là đã lên production.

8.1.3 Migration và dữ liệu

`prisma migrate deploy` chạy một lần trước đổi dịch vụ; API production không tự seed hoặc chạy migration mỗi replica. Seed chỉ dùng local/demo. Trước thay đổi schema phá hủy, backup bằng pg_dump và diễn tập restore. Dùng migration expand/contract tương thích cả image trước và image mới.

8.1.4 Health và smoke

- `https://DOMAIN/` là frontend.
- `https://DOMAIN/backend/health` là health của NestJS qua Caddy.
- `https://DOMAIN/backend/products` là API công khai theo đường dẫn gateway.
- `https://DOMAIN/api/products` là proxy Next cùng origin.
- `docker compose --env-file .env -f docker-compose.production.yml ps` kiểm tra container; `logs --tail=100 api` kiểm tra lỗi. Không log env, body thẻ, password, token.

8.1.5 Rollback

`deploy/deploy.sh SHA` lưu `.deployed-tag` chỉ sau smoke thành công. Nếu deploy/migration/smoke lỗi, script phục hồi image tag trước và trả exit 1. Không tự rollback schema: image trước chỉ an toàn nếu migration tương thích ngược. Nếu migration phá hủy hoặc dữ liệu hỏng, dừng nhận giao dịch và phục hồi backup theo quyết định vận hành. Lần deploy đầu không có previous tag; script báo rõ, không giả vờ đã rollback.

8.1.6 CI và branch protection

Quality Gate chạy PR/push main, Postgres service test, unit/integration/build và E2E desktop/mobile. Yêu cầu GitHub bảo vệ main với check `quality`, không cho merge khi check đỏ. Bằng chứng một lần lỗi và lần thành công cùng link PR nằm ở quality/verification.md. Cấu hình YAML không tự tạo môi trường production hoặc bật branch protection.

8.1.7 Backup mẫu

Từ thư mục production, chạy `docker compose --env-file .env -f docker-compose.production.yml exec -T db pg_dump -U brewlite brewlite > backup.sql`. Lưu backup ở nơi truy cập hạn chế, thử restore vào DB tách biệt. Không commit backup chứa dữ liệu người dùng.

8.2 Kịch bản demo trực tiếp

Thời lượng gợi ý 7–10 phút. Chạy Compose và mở web/API health trước demo; đây là kịch bản thực hiện, không phải link video đã quay.

1. Giới thiệu Next.js/NestJS/PostgreSQL, Product Backlog và ma trận truy vết. Cho xem health và menu API (1 phút).
2. Trên viewport điện thoại, mở Cappuccino, chọn M + Trân châu, thêm giỏ. Đổi qty rồi reload để chứng minh persist (1 phút).
3. Đăng ký tài khoản mới hoặc dùng demo, áp WELCOME10 cho đơn đủ 50k, chọn WALLET; xác nhận PAID và điểm đúng floor(total/10000) (1 phút).
4. Mở lịch sử, mã đơn và tiến trình. Đăng nhập barista ở cửa sổ khác, chuyển PAID→PREPARING→READY→COMPLETED; chờ polling cập nhật (1 phút).
5. Tạo đơn khác, CARD 4000000000000002; xem PAYMENT_FAILED và giỏ còn nguyên; đổi thẻ 4242424242424242; trả lại thành công trên cùng mã đơn (1 phút).
6. Mở test PAY-T03 cho 10 payment cùng key và INT-T09 cho 20 order tranh 5 phần; trình bày kết quả thật. Thử transition sai và xem 409 (1 phút).
7. Cho xem PR/Actions lỗi→sửa→xanh, Docker images, README, LaTeX/PDF và sơ đồ. Phân biệt local/CD cấu hình với production đã deploy. Ghi nhận feedback thật của giảng viên thành issue (1–2 phút).

Video ghi màn hình luồng end-to-end trên ứng dụng chạy thật: `demo/BrewLite_Demo.webm` (không lồng tiếng). Kịch bản trên dùng cho demo trực tiếp có giải thích.

8.3 Release notes

8.3.1 Phiên bản 1.0.1 ngày 07 tháng 11 năm 2026

Chuyển repository nộp bài sang https://github.com/Kenkra9999/Brew_Lite_2 với nguyên lịch sử commit và tag `v1.0.0`. Tạo lại Product Backlog bằng 15 GitHub Issue chia milestone Sprint

1/Sprint 2, bật branch protection cho `main` và kiểm chứng lại trên repository mới: Quality Gate xanh trên `main`, Delivery xuất image GHCR theo SHA, PR cố ý lỗi lint bị chặn merge. Cập nhật README, tài liệu bằng chứng và báo cáo LaTeX theo repository mới. Không thay đổi mã nguồn ứng dụng; các giới hạn của bản 1.0.0 vẫn giữ nguyên.

8.3.2 Phiên bản 1.0.0 ngày 06 tháng 11 năm 2026

Hoàn thiện luồng menu đến xác nhận: Next.js mobile/PWA, NestJS REST, Prisma migration PostgreSQL, JWT/bcrypt, size/topping, cart persist, WALLET/CARD mock, history/tracking và staff board. Bổ sung promo/loyalty, state machine, optimistic kho, payment key và khóa chống race hủy/thanh toán. Có unit/integration/E2E, Compose local, Quality Gate và workflow xuất image/CD, UML và báo cáo LaTeX.

Giới hạn phiên bản: thanh toán mô phỏng, chưa có gateway thật/reconciliation; kho PENDING không tự hết hạn; chưa có create-order idempotency; seed demo không dùng production; chưa cấp host/domain nên production chỉ có cấu hình. Danh sách thành viên và biên bản nghiệm thu cần nhóm bổ sung. Xem verification.md để biết chính xác test và CI đã thực sự chạy.

Chương 9

Truy vết và đối chiếu tiêu chí

9.1 Ma trận truy vết

Đường dẫn code tương đối từ `apps/api/src` hoặc `apps/web/src`. Tên test trỏ đến test thực trong repository. NFR chưa có bằng chứng phải giữ trạng thái chưa nghiệm thu; không suy ra hoàn thành từ việc có mã nguồn.

Yêu cầu	Use case / story	Thiết kế	Code hoặc API	Test / bằng chứng
FR-01	UC-01 / US-02,03	usecase-system, activity-checkout	products.service.ts; GET /products; web/app/page.tsx	products.int-spec.ts; WEB-T01..03; E2E-T01
FR-02,03	UC-01 / US-04	class-domain, activity-checkout	GET /products/:id; products/[id]/page.tsx; pricing.ts	products.int-spec.ts; pricing.spec.ts; E2E-T01
FR-04..06	UC-02 / US-05	activity-checkout	web/lib/cart.ts; cart/page.tsx	CART-T01..05; E2E-T01 reload
FR-07,08	UC-03 / US-07	usecase-system; class-domain	auth.controller/service.ts; POST /auth/register, /auth/login	auth.int-spec.ts; E2E-T01,02
FR-09,10	UC-03,04 / US-06,07	sequence-checkout	auth/guards.ts; order.dto.ts; bootstrap.ts	INT-T01,02,06,07; PAY-T07
FR-11	UC-04 / US-10	class-domain; sequence-checkout	promotions.service.ts; POST /promotions/validate	promotion-rules.spec.ts; INT-T04; PAY-T01
FR-12	UC-04 / US-06	sequence-checkout; class-domain	orders.service.ts; POST /orders	INT-T03,04,05,09; E2E-T01
FR-13,14	UC-04 / US-08	sequence-checkout; state-order	payments.service.ts; POST /payments; checkout/page.tsx	PAY-T01,05,07,10; E2E-T01,02
FR-15,16	UC-05 / US-09	activity-checkout; class-domain	GET /orders/me, /orders/:id; web/app/orders	INT-T06,07; E2E-T01
FR-17	UC-06 / US-10	state-order; class-domain	order-state-machine.ts; PATCH /orders/:id/status; web/app/staff	order-state-machine.spec.ts; INT-T07

Yêu cầu	Use case / story	Thiết kế	Code hoặc API	Test / bằng chứng
FR-18	UC-04 / US-10	sequence-checkout	orders.service.ts createInTx; Prisma Product.version	INT-T09: 20 concurrent requests / 5 stock and qty=2
FR-19	UC-04 / US-10	class-domain; sequence-checkout	loyalty.ts; Payment finalize	loyalty.spec.ts; PAY-T01,02,08
FR-20	UC-04,06 / US-10	state-order	POST /orders/:id/cancel; orders.service.ts transition	INT-T08; PAY-T08,09
FR-21	UC-04 / US-08,10	sequence-checkout	payments.service.ts pay/ replay; UNIQUE Payment.idempotencyKey	PAY-T02,03,04,06
FR-22,23	Ngoài MVP / backlog	usecase-system	Không hiện thực cổng thật hoặc push notification	Không áp dụng nghiệm thu MVP
NFR-01	Hiệu năng	deployment	GET /products	tools/ measure-performance.mjs; quality/performance.json
NFR-02	Bảo mật	deployment; sequence-checkout	bcrypt/JWT/DTO; Compose DB private	auth.int-spec.ts; INT-T01,02,06,07; PAY-T07
NFR-03,05	Khả dụng và tái lập	deployment	GET /health; Dockerfiles; migrations; Compose	Compose health và build; migration reset integration
NFR-04,08	Mobile và truy cập	activity-checkout	web/globals.css; labels/ form/error	E2E desktop + mobile; screenshot kiểm tra bố cục
NFR-06	Quality gate	deployment CI	ci.yml; branch protection main	Actions runs/PR được liên kết trong verification.md
NFR-07	Production	deployment production	delivery.yml; deploy.sh; compose production	Cần host/domain, secrets, URL thật và smoke trước khi Done
NFR-09	PWA	deployment browser	manifest.webmanifest; sw.js; offline.html	Kiểm tra tệp và service worker; cài đặt cần HTTPS/ localhost

9.2 Checklist bàn giao theo rubric

9.2.1 Assignment 1

Task	Tạo phẩm	Trạng thái
1	Monorepo Next/Nest, env mẫu, README, Git	Có mã nguồn và lịch sử
2	GET /products từ PostgreSQL seed	Có API và integration test
3	Menu mobile loading/empty/error	Có UI và component test
4	Chi tiết, size/topping, tính giá	Có UI, pricing test và E2E

Task	Tạo phẩm	Trạng thái
5	Giỏ thêm/sửa/xóa, badge/tổng, persist	Có store test và E2E reload
6	POST /orders DTO, PENDING	Có transaction và integration test
7	JWT register/login bcrypt guard	Có integration test phân quyền
8	Ví/Thẻ mock PAID/PAYMENT_FAILED	Có success/failure/retry test
9	Xác nhận/lịch sử/Compose/README/demo	Có mã nguồn, Docker và kịch bản demo trực tiếp
10	State/idempotency/kho đồng thời/promo/loyalty	Có unit và integration chứng minh

9.2.2 Assignment 2

Task	Tạo phẩm	Trạng thái
1	context-and-scope, SRS MoSCoW, use case	docs/requirements và use-case diagrams
2	Activity/sequence/class, nguồn và kết xuất	docs/diagrams .puml/.svg/.png
3	Architecture, ADR, deployment diagram	Có thiết kế; production topology cần cập nhật host thật
4	Repo, backlog, menu MVP, CI, merge gate	Repository Brew_Lite_2, 15 issue theo milestone Sprint 1/2, PR lỗi bị chặn và PR xanh được merge; xem verification.md
5	End-to-end, tests, Compose, images/CD, production, release	Phần local/test/CI có bằng chứng; production cần host/domain/secrets và URL hoạt động

9.2.3 Thông tin nhóm cần bổ sung trước nộp

- Repository nộp bài: https://github.com/Kenkra9999/Brew_Lite_2 (public); cấp quyền/gửi link cho giảng viên.
- Điền nhóm, lớp, thành viên/MSSV trong report/thong-tin.tex; xác nhận tên GVHD theo mẫu.
- Tổ chức demo/Review với giảng viên, ghi feedback và vai trò Scrum thực.
- Cung cấp môi trường production, deploy, cập nhật URL frontend/API/health và sơ đồ thực tế.
- Có thể bổ sung video quay demo nếu không chọn demo trực tiếp.

Theo đề A2, thiếu URL production hoạt động thì phần triển khai Task 5 chưa được tính hoàn thành. Báo cáo và repository giữ rõ giới hạn này.

Chương 10

Tài liệu tham khảo

10.1 Tài liệu tham khảo

1. Đề BrewLite Công nghệ Phần mềm, version 1.0, Đại học Sài Gòn, HK I 2026–2027, file BrewLite-CongNghePhanMem.pdf do người học cung cấp.
2. Đề BrewLite Assignment 2 Requirements to Deployment, version 1.0, SGU, file Assignment2-BrewLite.pdf do người học cung cấp.
3. Mẫu Assignment_report.tex và logo trong Assigment_template_SGU được cung cấp cùng đề; dùng bố cục nhận diện SGU, cập nhật học kỳ/năm học theo đề mới.
4. Next.js Route Handlers: https://nextjs.org/docs/app/api-reference/file-conventions/route — tham khảo Web Request/Response và Promise params; triển khai thực tế kiểm tra với Next 15.5.29 đã khóa trong package-lock.
5. Docker Compose startup order: https://docs.docker.com/compose/how-tos/startup-order/ — healthcheck và condition service_healthy.
6. PlantUML: https://plantuml.com/ — nguồn UML và local renderer 1.2026.2.
7. Prisma transactions: https://www.prisma.io/docs/orm/prisma-client/queries/transactions — transaction, optimistic concurrency và idempotency.
8. GitHub Actions: https://docs.github.com/en/actions — pipeline và registry; file workflow trong repo là thiết kế triển khai cụ thể.

Tham khảo ngày 06/11/2026. Không sao chép lời mô tả thư viện làm minh chứng hệ thống đã deploy; kết quả thực nghiệm được lưu ở quality/verification.md.